Relatório do Trabalho Prático

Sistemas Operativos

2025/2026



Realizado por:

Martim Oliveira Jardim Freitas – 2021145193

Rodrigo Gonçalves Glória – 2022150127

1. Introdução

O presente relatório descreve a conceção e implementação de um sistema de simulação para a gestão de uma frota de táxis autónomos, desenvolvido no âmbito da unidade curricular de Sistemas Operativos.

O objetivo principal do projeto foi o desenvolvimento de uma plataforma robusta em linguagem C para ambiente UNIX, capaz de gerir múltiplos pedidos de transporte e coordenar veículos em tempo real. A arquitetura do sistema divide-se em três processos distintos que interagem entre si.

- Controlador: O componente central responsável pela gestão da frota, atribuição de serviços e interação com múltiplos clientes;
- Cliente: A interface que permite aos utilizadores agendar viagens, consultar estados e receber notificações;
- Veículo: Um simulador da viatura que reporta a sua telemetria e interage com o passageiro.

Para a concretização deste projeto, foram aplicados os conhecimentos adquiridos sobre a API do sistema operativo *Linux*, com foco especial na manipulação de processos, *multi-threading* e mecanismos de comunicação entre processos, nomeadamente *named pipes* e sinais.

2. Arquitetura e Estratégia de Implementação

A solução foi desenhada com base numa arquitetura do tipo Cliente-Servidor, onde o Controlador atua como o servidor central de gestão e os Clientes e Veículos operam como processos satélites.

2.1. Gestão de Processos e Threads

A arquitetura do sistema baseia-se num modelo de processamento multitarefa. No Controlador, optou-se por um modelo *Multi-Thread* (utilizando a biblioteca '*pthread*'). Esta escolha permitiu processar pedidos de múltiplos clientes, gerir a interface de administração e simular a passagem do tempo simultaneamente, partilhando o mesmo espaço de endereçamento e estruturas de dados globais.

Para a simulação da Frota, optou-se pela criação dinâmica de processos independentes. Cada veículo é instanciado através da chamada ao sistema *fork()* seguida de *exec()*. Isto garante que cada veículo opera isoladamente; se um processo veículo falhar, não compromete a estabilidade do Controlador.

2.2. Mecanismos de Comunicação

A comunicação entre os diferentes processos recorre exclusivamente aos mecanismos nativos do sistema operativo.

- Named Pipes (FIFOs) – Utilizados para a comunicação assíncrona entre processos não relacionados.
    - Existe um único pipe de entrada conhecido (controlador_fifo) onde o Controlador.c recebe os pedidos;
    - Cada cliente possui um pipe exclusivo (pipe <PID>) para receber respostas do servidor e notificações diretas dos veículos (Exemplo: Chegada ao local de recolha do passageiro), assegurando a troca direta de mensagens

entre a viatura e o cliente, sem intervenção do Controlador.

- Pipes Anónimos (Unnamed Pipes) – Utilizados exclusivamente na comunicação Pai-Filho entre o Controlador e o Veículo.
    - Ao lançar um veículo, o controlador redireciona o *stdout* do processo filho para um *pipe anónimo*. Isto permite ler a telemetria (percentagem do progresso) de forma contínua e eficiente.

- Sinais – Utilizados para o controlo de fluxo prioritário. O sinal SIGUSR1 é empregue para ordenar o cancelamento imediato de viagens ou o encerramento dos clientes aquando do fecho do sistema.

2.3. Sincronização e Integridade de Dados

Dada a natureza concorrente do Controlador, a integridade dos dados partilhados é assegurada através de mecanismos de exclusão mútua (*Mutexes*). Foi implementada uma estratégia, com mutexes distintos, para proteger diferentes secções críticas: Enquanto os mutexes *'m_frota'* (estado dos veículos), *'m_agenda'* (lista de pedidos) e *'m_clientes'* (sessões ativas) salvaguardam a consistência integral das estruturas de dados do sistema, os mutexes *'m_km'* e *'m_tempo'* destinam-se a garantir a atualização segura das variáveis globais de estatística e relógio. Esta abordagem minimiza o tempo de bloqueio e evita que uma operação de consulta bloqueie a admissão de novos utilizadores.

2.4. Robustez e Tratamento de I/O

Para melhorar a experiência de utilização, o Cliente recorre à criação de um processo filho dedicado à receção de mensagens. Este processo utiliza a chamada de sistema *select()* para monitorizar

simultaneamente a chegada de dados pelo *Named Pipe* (respostas do servidor/veículo) e o estado do Pipe Anónimo de monitorização (ligado ao processo pai). Esta arquitetura permite receber notificações em tempo real sem bloquear a escrita de comandos no teclado, garantindo ainda que o processo de leitura termina automaticamente caso a interface principal seja encerrada.

3. Análise de Requisitos e Funcionalidades

Módulo	Funcionalidade	Estado	Obs. / Detalhes de Implementação
Geral	Compilação via Makefile	**Cumprido**	Targets definidos para todos os executáveis.
	Arquitetura Cliente-Servidor	**Cumprido**	Separação clara de processos e responsabilidades.
Cliente	Identificação única (Login)	**Cumprido**	Validação de usernames duplicados no servidor.
	Envio/Receção simultânea	**Cumprido**	Uso de select() para I/O não bloqueante.
	Agendar viagem (agendar)	**Cumprido**	Validação de disponibilidade e envio ao servidor.
	Consultar serviços (consultar)	**Cumprido**	Visualização de estados (Pendente / A Decorrer).
	Cancelar serviços (cancelar)	**Cumprido**	Cancelamento por ID ou global.
	Interação com Veículo	**Cumprido**	Recebe notificações diretas (Início/Fim de viagem).
Controlador	Gestão de Frota	**Cumprido**	Limite configurável via NVEICULOS.
	Lançamento de Veículos	**Cumprido**	Instanciação dinâmica via fork e exec.
	Receção de Telemetria	**Cumprido**	Leitura contínua do stdout via pipe anónimo.
	Interface de Administração	**Cumprido**	Comandos de gestão e monitorização implementados.
	Contabilização de Kms	**Cumprido**	Variável global protegida por Mutex.
	Encerramento do Sistema	**Cumprido**	Notifica clientes e elimina processos de forma ordeira.
Veículo	Simulação de Trajeto	**Cumprido**	Avanço temporal e cálculo de percentagem.
	Comunicação de Estado	**Cumprido**	Escrita no stdout para o controlador ler.
Extras	**Negociação de Horário**	**Extra (Comando Decisão)**	Sistema propõe nova hora se a frota estiver cheia.

4. Conclusão

O desenvolvimento deste projeto permitiu a implementação bem-sucedida de um sistema de gestão de frota, consolidando na prática os conceitos fundamentais de Sistemas Operativos. A solução apresentada cumpre integralmente os requisitos funcionais, destacando-se pela robustez da arquitetura *multi-thread* do Controlador e pela gestão eficiente da comunicação entre processos (IPC).

As principais dificuldades, inerentes à sincronização de tarefas simultâneas e à partilha de recursos, foram superadas através de um rigoroso controlo de concorrência, o que garantiu a estabilidade do sistema. Em suma, o sistema final não só responde corretamente à interação simultânea de múltiplos utilizadores, como implementa mecanismos de tolerância a falhas que asseguram a integridade operacional da simulação.